Idea 2: Per-Context Thresholds for AI-Agent Monitors

Heterogeneity, CFAR vs global vs Stackelberg allocation, attack selection, and baseline poisoning

October 2026

Summary. Honest monitor-score distributions differ across contexts, so one global threshold over-audits some contexts and under-audits others. Radar's answer is CFAR: hold the false-alarm rate constant in every context. We tested CFAR and alternatives on real monitor data (Gemini-3.1-Pro streaming judges on 7 settings, the 12-monitor LinuxArena sweep, MRT, MonitoringBench) at a fixed *global* audit budget, against uniform, context-selecting and poisoning attackers.

1. **Heterogeneity is real across deployments, small within one.** When one monitor and rubric serve different deployments, a single global 5% threshold gives 0–9% per-setting FPR (random-effects $\tau = 1.34$ logit, $p < 0.001$) and 0–16% across setting×agent. Within LinuxArena it is 2–7% across environments, and agent model does not matter.
2. **CFAR is the worst rule against an attacker who chooses where to attack.** Safety at a 1% budget is 0.40 vs 0.64 for global on 7 settings, and 0.22 vs 0.60 on ControlArena. CFAR hands the attacker the loose contexts.
3. **Damage-equalizing (Stackelberg) thresholds are best against attack selection.** Safety is 0.79 vs 0.64 at 1% and 0.97 vs 0.89 at 5%, at ≤ 0.02 cost against a uniform attacker.
4. **Per-context quantiles break with few honest runs.** At $n = 10$ per context they spend 7× the budget. Partial pooling with cross-validated strength stays near budget.
5. **CFAR recalibration is easy to poison.** 2% poisoned calibration drops the target context's TPR from 0.95 to 0.37. A robust order-statistic CFAR with its tail estimated from the other contexts holds 0.77–0.86 and still tracks genuine drift.

Spend: $17.11 OpenRouter (trajectory-mode re-scoring of MonitoringBench).

Contents

1 Question **2**

2 Data and contexts **2**

3 Methods **3**

3.1 Heterogeneity . . . 3

3.2 Allocation rules . . . 3

3.3 Evaluation . . . 4

3.4 Poisoning . . . 4

4 Results **5**

4.1 Heterogeneity . . . 5

4.2 Allocation at a fixed global budget 6
4.3 Small samples per context 8
4.4 Poisoning 9

5 Recipe **11**

6 Limitations and negative results **11**

1 Question

How much do honest monitor-score distributions vary by context (environment, setting, task type, agent model, monitor)? At a **fixed global audit budget** B (fraction of honest traffic audited), do per-context thresholds beat a single global threshold? How do they compare with a detection-maximizing allocation and a damage-equalizing Stackelberg allocation:

- against a uniform attacker;
- against an attacker who knows the thresholds and chooses its context;
- against an attacker who poisons its own context's honest calibration data?

And do robust estimators defend?

Prior expectations. Per-context thresholds are requested by several research agendas: *Salient Directions*, Scale MRT, UK AISI Async Control, and GDM's UEBA mention. Pinto (2607.06596) shows a 1% threshold transferred across model families yields 0–94% realized audit spend. But detection theory (Ghafouri et al.) predicts that against a threshold-aware attacker the optimal per-context thresholds equalize worst-case *damage*, not FPR. CFAR may therefore be exploitable.

2 Data and contexts

Each *panel* is a dataset × monitor × context key. The trajectory statistic is the max of per-step (or per-chunk) scores.

Panel	Monitor (scale)	Context (#)	Honest / attack	Honest per ctx
la_env (local / cum.)	Gemini-3.1-Pro streaming (aksh-n LinuxArena gold set; 0–10)	environment (10)	1,396 / 1,305	87–197
la_taskcat	same	task category (5)	1,332 / 1,296	75–620
la_model	same	agent model (8)	1,155 / 1,298	45–583
akshn_setting ("7 settings")	Gemini-3.1-Pro local, same rubric	setting (7)	5,189 / 5,794	328–1,396
ca_setting_model	same	ControlArena setting × agent (12)	2,997 / 3,000	247–250
lasweep_{m}	12 LinuxArena sweep monitors (0–100)	environment (10)	200 / 101–142	20
mrt_{m}	claude-3.5-sonnet sequential $N = 5$, 3 prompts (1–5)	CUA task (15)	150 / 675	10
mb_{m}	gpt-4.1-mini / nano (trajectory and per-step)	BashArena task cluster (7, TF-IDF k-means)	535 / 752–2,584	30–207

The Gemini panels are strong monitors (AUROC ≈ 0.99); the MonitoringBench panels are weak (AUROC 0.59–0.76). BashArena "honest_summarized" rows are excluded.

3 Methods

3.1 Heterogeneity

- One-way random-effects ANOVA of honest trajectory max (ICC), and nested ANOVA of per-step scores.
- Per-context realized FPR under one traffic-weighted global threshold at 1/5/10% (Wilson CIs).
- A logit-normal random-effects model of per-context exceedance counts. Its τ is the SD of logit-FPR net of binomial noise, with an LR test of $\tau = 0$. Reported at 5% and 10%; not identifiable at 1%.
- Cross-monitor agreement of LinuxArena environment effects.

3.2 Allocation rules

All rules are budget-matched with randomized thresholds, so calibration spend $\sum_c w_c \mathrm{FPR}_c = B$ exactly (w_c = traffic share).

Rule	Definition
global	one traffic-weighted randomized quantile
CFAR	per-context randomized plug-in quantile (constant FPR per context)
Mondrian	per-context conformal quantile (finite-sample valid)
pooled_eb / pooled_cv	per-context quantile of $(n_c\hat{F}_c + n_0\hat{F})/(n_c + n_0)$; n_0 by beta-binomial empirical Bayes or 5-fold CV
maxdet	fractional knapsack on per-context ROC hulls maximizing uniform-attacker TPR
Stackelberg	randomized thresholds on ROC hulls minimizing $\max_c D_c(1 - \mathrm{TPR}_c)$ (bisection on the payoff); $D_c \equiv 1$, or $D_c \sim \log U[1, 10]$ for the damage-weighted variant
OS-CFAR / TM-CFAR	per-context robust location (p75 offset / 10%-trimmed mean, winsorized SD) with a common tail level

3.3 Evaluation

- **Splits.** 60 stratified 50/50 splits (point estimates) and 150 split-then-bootstrap replicates (95% intervals; paired differences vs global). Honest runs are split by context; attacks by context × attack type.
- **Attackers:**
  - *uniform*;
  - *informed selection*: picks the context with the highest undetected-success probability, estimated from its own calibration attacks under the published thresholds;
  - *threshold-only*;
  - *oracle*;
  - *cell-level*: picks a context × attack-type cell;
  - *damage-weighted*.
- **Two reporting modes:**
  - *nominal*;
  - *spend-matched*: the nominal level is swept over $B \times [0.25, 2]$ and metrics are interpolated at test spend $= B$. This is an evaluation device only.
- **Small samples.** Calibration is subsampled to $n \in \{10, 30, 100\}$ honest runs per context.

3.4 Poisoning

- **Splits.** Honest runs per context are split into a trusted reference, a new window, and test (1/3 each).
- **Attack.** The attacker adds a fraction ε of the target's new window at value v: either the attacker's own maximum calibration attack score, or the panel's honest maximum (plausibly benign). Every context is targeted in turn, 40 replicates.
- **Boiling frog.** 30 recalibration rounds with a 10-round sliding window, $B = 1\%$. The attacker injects ε just above the current threshold. A genuine-drift control shifts the target by +2 score points from round 10.
- **Defenses:**
  - global; CFAR; pooled_eb;
  - OS/TM-CFAR with a leave-one-context-out (LOCO) tail;

– 50/50 reference mix; frozen reference;
– drift cap (reference +10% of range);
– outlier filter; cumulative cap.

4 Results

4.1 Heterogeneity

Panel	ICC (max)	Per-context FPR at global 5%	τ at 5% (p)
7 settings	0.17 (0.31 normal scores)	0–9.0%	1.34 (< 0.001)
ControlArena setting × agent	0.28	0–15.8%	1.75 (< 0.001)
LinuxArena environment	0.05	2.1–7.2%	0.14 (0.38); 0.45 at 10% (< 0.001)
LinuxArena task category	0.04	3.7–12.8%	0.40 (0.04)
LinuxArena agent model	0.00	2.1–8.0%	0.19 (0.28)
LinuxArena sweep (20 honest/env)	0.05–0.31	0–25%	0–1.46
MRT (10 honest/task)	0.06–0.46	0–49%	0–1.94
BashArena clusters (gpt-4.1 per-step)	≤ 0.03	0–12%	n.s.

- Context explains only 0.1–9% of *per-step* variance (MRT 8–38%). The heterogeneity lives in the trajectory tails.
- **Agent model is not a useful context.**
- LinuxArena environment effects are shared across all 12 monitors: mean pairwise Spearman 0.70 (range 0.43–0.97). The environment×monitor interaction is no larger than noise (common-effect variance 0.148 vs cell noise 0.037). The heterogeneity therefore comes from the *trajectories* (some environments are intrinsically sensitive-looking), not from monitor quirks.

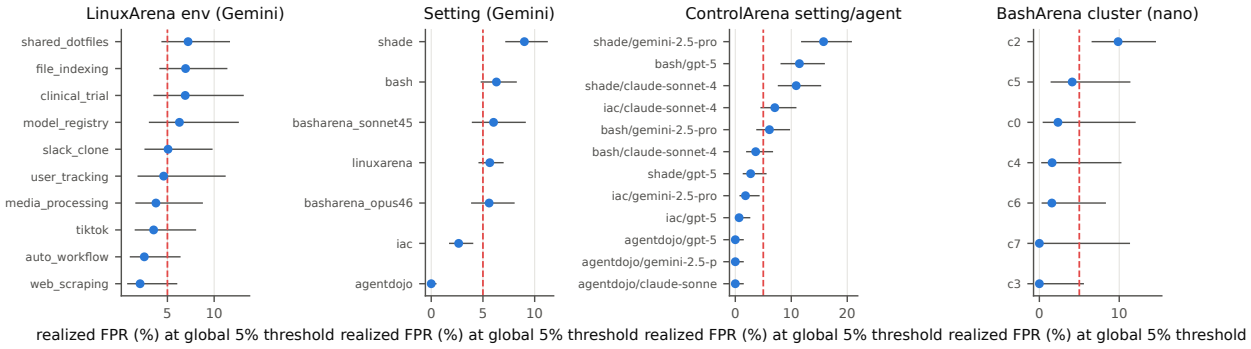


Figure 1: Realized per-context FPR under a single global 5% threshold (Wilson 95% CIs), for four panels.

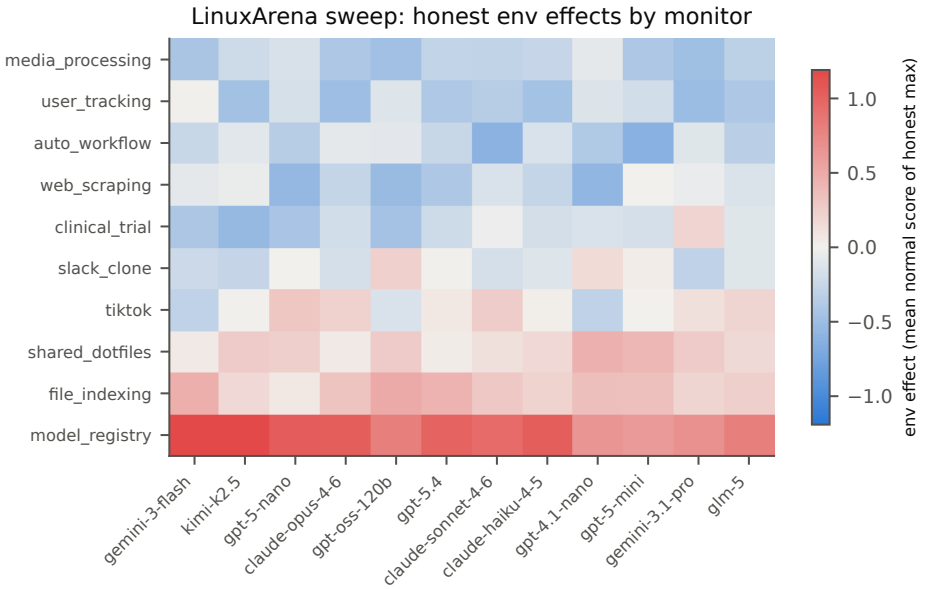


Figure 2: LinuxArena sweep: standardized honest environment effects by monitor. The same environments run hot for every monitor.

4.2 Allocation at a fixed global budget

Panel, B	Rule	Spend	Uniform TPR	Selection safety	Δ vs global (95%)
7 settings, 1% (nominal)	global	1.01%	0.853	0.645 [0.53, 0.70]	–
	CFAR	1.13%	0.821	**0.404** [0.18, 0.56]	$-0.47..-0.10$
	max-detect	1.12%	**0.891**	0.641 [0.47, 0.86]	$-0.09..+0.21$
	Stackelberg	1.07%	0.834	**0.791** [0.69, 0.89]	$+0.06..+0.24$
	Mondrian	0.36%	0.689	0.125	–
	pooled_cv	1.10%	0.829	0.439	–
ControlArena, 1% (nominal)	global	0.97%	0.798	0.601	–
	CFAR	1.34%	0.837	**0.218**	$-0.58..-0.11$
	Stackelberg	1.38%	0.863	**0.729**	$-0.07..+0.27$
7 settings, 5% (spend-matched)	global	5%	0.965	0.890	–
	CFAR	5%	0.946	0.753	$-0.30..+0.01$
	max-detect	5%	**0.984**	0.958	$+0.02..+0.12$
	Stackelberg	5%	0.971	**0.966**	$+0.03..+0.12$
ControlArena, 5% (spend-matched)	global / CFAR / Stackelberg	5%	0.960 / 0.935 / 0.982	0.908 / 0.561 / 0.973	CFAR $-0.63..-0.06$

Table 1: "Selection safety" is the TPR in the context an informed attacker picks. Nominal-mode rules can over-spend out of sample. The spend-matched rows remove this; their conclusions are the same.

- **Damage-weighted Stackelberg** under a damage-weighted attacker (5%): 0.984 vs 0.937 (7 settings) and 0.989 vs 0.930 (ControlArena), $+0.01..+0.12$.
- **Homogeneous panels** (LinuxArena): all rules are within ± 0.02 uniform TPR and ± 0.1 selection safety of global. Allocation does not matter when contexts do not differ.
- **Weak monitors** (MonitoringBench clusters): CFAR and max-detect raise uniform TPR (nano trajectory at 5%: $0.079 \to 0.106$ / 0.162), but selection safety stays low under every rule.
- **Cell-level selection** (choosing an attack type within a context) costs another 0.2–0.4: LinuxArena environment at 1% falls from 0.90 to 0.61 (global) and from 0.95 to 0.70 (Stackelberg).

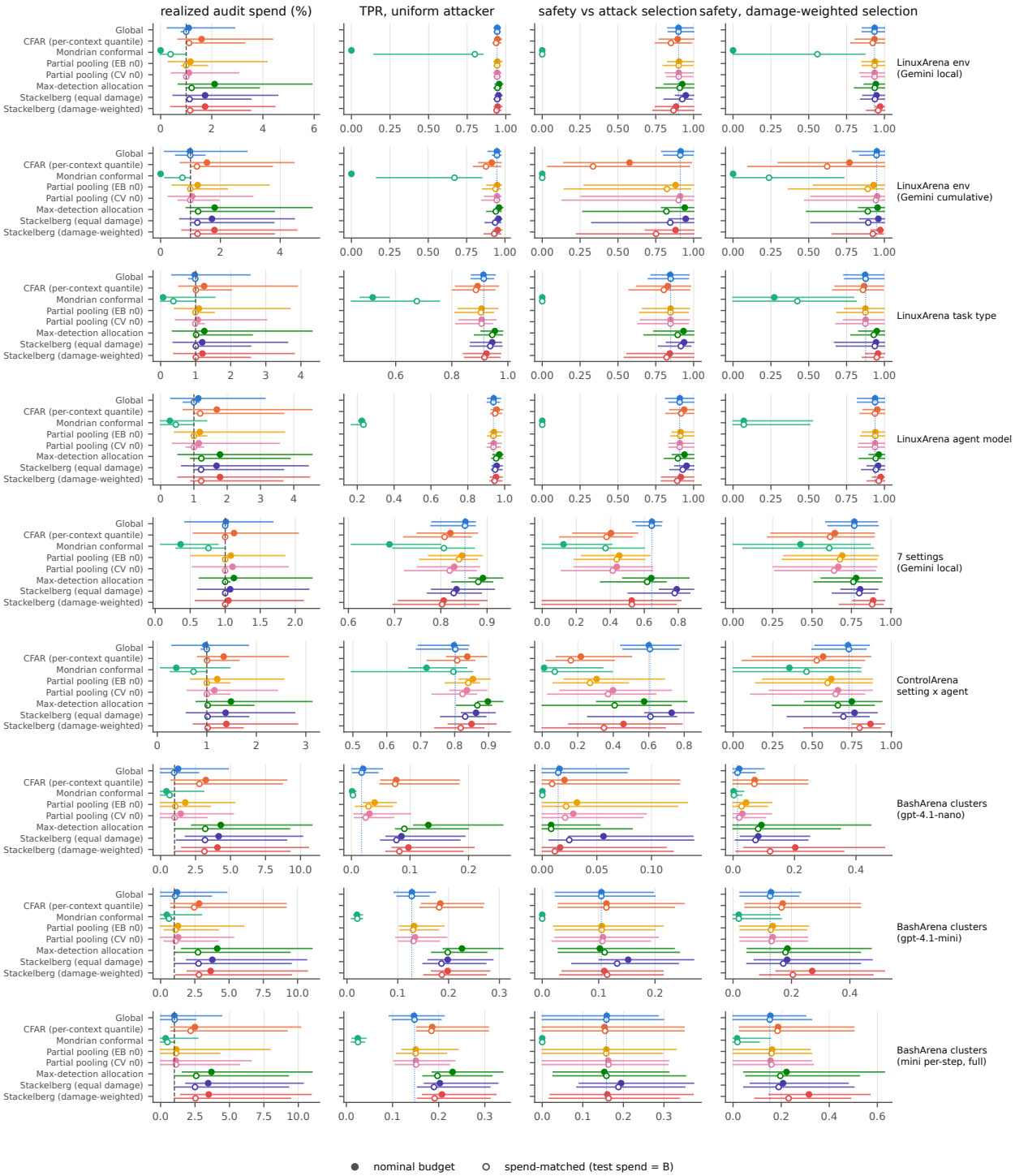


Figure 3: Allocation rules at $B = 1\%$, all panels. Columns: realized audit spend, uniform-attacker TPR, safety against attack selection, safety against damage-weighted selection (filled: nominal; open: spend-matched).

4.3 Small samples per context

n per ctx	global	CFAR	Mondrian	pooled_eb	pooled_cv
10	1.61× / 0.81 / 0.63	**7.26**× / 0.91 / 0.59	0 / 0 / 0	2.45× / 0.85 / 0.63	1.72× / 0.83 / 0.64
30	1.16× / 0.83 / 0.63	2.90× / 0.85 / 0.43	0 / 0 / 0	1.60× / 0.85 / 0.60	1.30× / 0.84 / 0.61
100	1.01× / 0.84 / 0.64	1.48× / 0.84 / 0.40	0.46× / 0.68 / 0.08	1.25× / 0.85 / 0.52	1.13× / 0.84 / 0.58

Table 2: Spend/B / uniform TPR / selection safety, 7 settings, $B = 1\%$.

- The plug-in per-context quantile spends $\approx 1/(n_c + 1)$ per context regardless of target.
- Mondrian conformal is valid but *never alarms* below $n_c \approx 1/B$.
- Partial pooling with cross-validated n_0 stays within 1.1–1.3× of budget (1.4–2.9× at $n = 10$) at equal TPR.
- On the LinuxArena sweep and MRT (10 calibration runs per context, nominal 5%): CFAR spends 7.8–12.3%, max-detect and Stackelberg 9–14%, global and pooled_cv 4.6–6.5%.

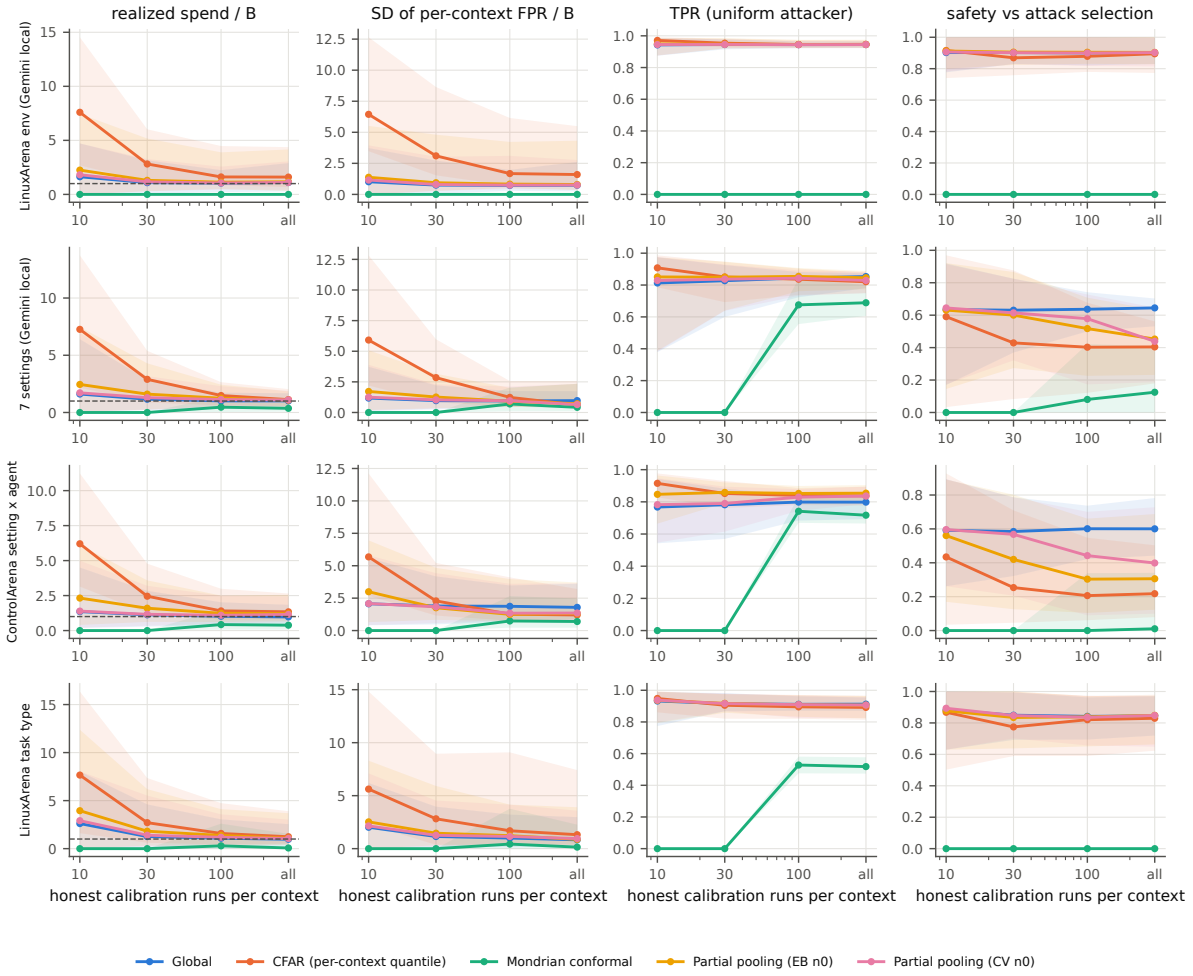


Figure 4: Small-sample regime: spend/B, SD of per-context FPR/B, uniform TPR and selection safety vs honest calibration runs per context.

4.4 Poisoning

Poisoned share ε	0	1%	2%	5%	10%	20%
global	.945	.942	.941	.933	.785	.415
CFAR	.949	.758	**.366**	.157	.079	.039
pooled_eb	.947	.940	.931	.759	.360	.072
OS-CFAR (LOCO)	.896	.890	.887	**.862**	**.807**	.383
50/50 reference mix	.943	.928	.724	.314	.157	.078
CFAR + drift cap	.953	.878	.800	.778	.771	.769
frozen reference	.945	.945	.945	.945	.945	.945

Table 3: TPR in the poisoned context (LinuxArena environment, $B = 1\%$, attack-max poison). The 7-settings panel is similar (CFAR 0.11, OS-CFAR 0.81 at 5%). TM-CFAR is dominated: clean TPR is 0.2–0.3 lower, and it breaks at its trim fraction.

- **CFAR breaks once $\varepsilon \approx B$.** A per-context quantile at level B is moved by any poison share $\geq B$.
- **Global breaks near $\varepsilon \approx B/w_t$.** The poison is diluted by the context's traffic share w_t.
- **OS-CFAR (LOCO)** takes its tail level from the *other* contexts, so a single poisoned context cannot move it. It holds TPR at 0.77–0.86 up to 10% poison.

Defense (boiling frog, $\varepsilon = 5\%$, 30 rounds)	TPR (LA / 7 settings)	FPR under genuine +2 drift
CFAR, no filter	0.16 / 0.11	2.0% / 1.1%
CFAR + outlier filter	0.16 / 0.11 (only slower)	4.6% / 2.7%
+ cumulative cap	0.78 / 0.54	4.9% / 3.3%
frozen-reference mix + filter	0.33 / 0.31	4.3% / 3.2%
global + filter	0.94 / 0.65	4.0% / 4.0%
OS-CFAR (LOCO) + filter	**0.82 / 0.80**	**1.1% / 0.6%**

Table 4: Gradual poisoning vs genuine drift. Only OS-CFAR with a leave-one-context-out tail is both poisoning-robust and drift-tracking.

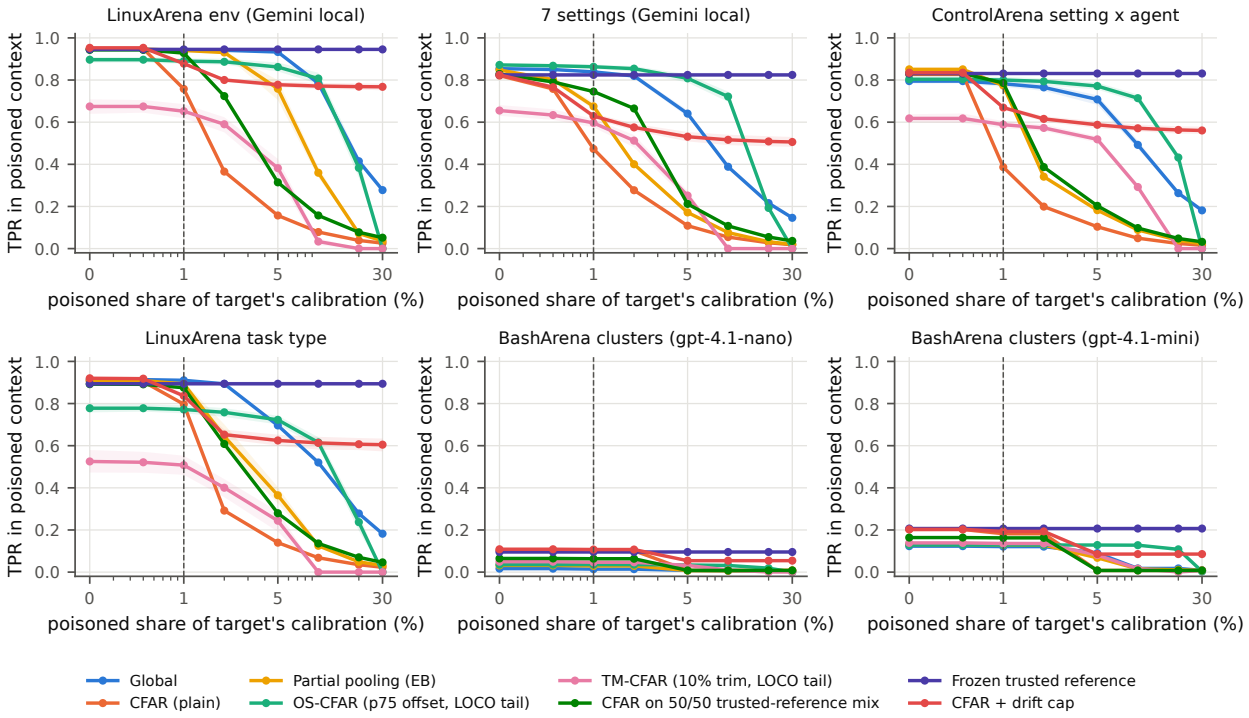


Figure 5: TPR in the poisoned context vs poisoned share of its calibration data, by defense and panel.

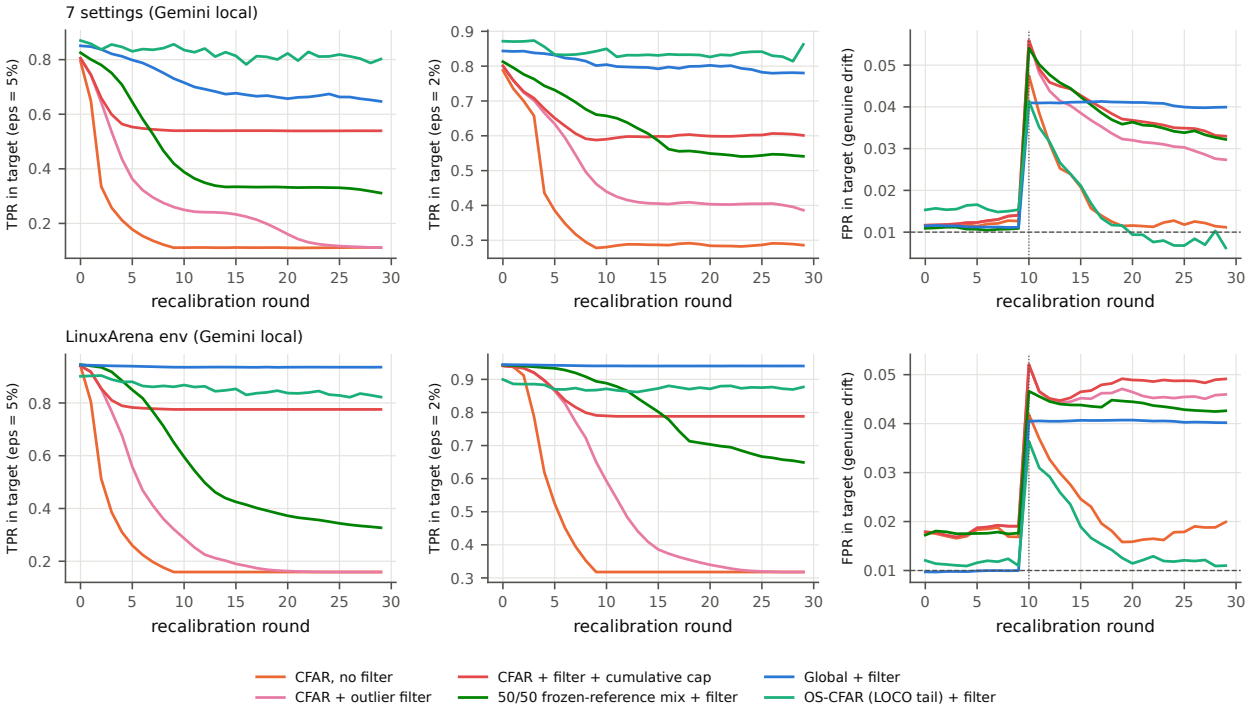


Figure 6: Boiling frog: TPR in the target context and FPR under genuine drift over 30 recalibration rounds.

5 Recipe

1. **Measure first.** Fit the random-effects τ on exceedances at a global 5–10% threshold. If τ is not significant, use one global threshold.
2. **Never use plain per-context quantiles with $n_c <\sim 3/B$.** Use partial pooling with cross-validated n_0, which falls back to global automatically. Mondrian is valid but useless below $n_c \approx 1/B$.
3. **If contexts truly differ and the attacker can choose where to act, do not use CFAR.** Use Stackelberg (damage-equalizing) thresholds fit on held-out attack data, with damage weights when stakes differ by context. Without attack data, use global. Max-detection is right only against a non-strategic attacker.
4. **If the policy can influence recalibration data**, use OS-CFAR with a leave-one-context-out tail, or global. Outlier filters only slow a boiling frog. Cumulative caps bound poisoning but over-spend under genuine drift.
5. **Expect within-context attack-type selection to cost more** (0.2–0.4) than allocation can recover.

6 Limitations and negative results

- **Monitor strength.** The Gemini panels are near-separable; the MonitoringBench monitors are weak.
- **Few attacks per context in the LinuxArena sweep** (5–18), so Stackelberg and max-detect overfit there.
- **Attacker modeling.** The informed attacker uses its own calibration-split scores. The oracle attacker is biased low (winner's curse). Damage weights are synthetic.
- **Untested.** Transfer of per-context thresholds across datasets. Poisoning across several contexts, which would weaken the global and LOCO defenses.
- **Negative results:**
  - TM-CFAR is dominated;
  - empirical-Bayes n_0 is worse than cross-validated n_0;
  - agent model is not a useful context;
  - trajectory-mode gpt-4.1-mini/nano are weak monitors (AUROC ≈ 0.6).

Reproduction

Code: `agent-monitor-detection/tracks/t2_per_context/` (`t2lib.py`, `panels.py`, `r1_heterogeneity.py`, `r1_allocation.py`, `r2_poisoning.py`, `r2_boiling.py`, `make_tables.py`, `figures.py`). Results: `results/t2/` (`tables.md`, CSVs, PDF figures). Repository: https://github.com/gregkocher/agent-monitor-detection (private).